

Основные паттерны в Qt6

1. Model-View-Controller (MVC) и его вариация Model-View

Где используется

Виджеты отображения данных (QListView, QTableView, QTreeView и др.).

Описание

Разделение данных, представления и логики управления. В Qt View и Controller часто объединены (Model-View архитектура):

- **Model** (QAbstractItemModel, QAbstractListModel, QAbstractTableModel) — хранит данные или ссылку на них, предоставляет стандартизированный API.
- **View** (QListView, QTableView) — запрашивает данные у модели и отображает их.
- **Delegate** (в Qt заменяет Controller) — отвечает за редактирование и отображение отдельных элементов.

Пример: отображение списка задач в QListView с моделью QAbstractListModel.

2. Signal-Slot (сигналы и слоты)

Где используется

Повсеместно для коммуникации между объектами.

Описание

Механизм асинхронного взаимодействия:

- **Сигнал** — событие, которое испускает объект (например, нажатие кнопки).
- **Слот** — метод, который вызывается при получении сигнала.

Примеры:

- подключение кнопки (QPushButton) к функции через connect();
- обработка изменений в полях ввода (QLineEdit::textChanged).

3. Observer (наблюдатель)

Где используется: тесно связан с паттерном Signal-Slot.

Описание

Объекты подписываются на изменения других объектов. В Qt это реализуется через сигналы:

- виджеты наблюдают за моделями (например, `QTableView` следит за `QStandardItemModel`);
- изменения в модели автоматически отражаются в представлении.

4. Factory Method (фабричный метод)

Где используется: создание UI-элементов и ресурсов.

Описание

Делегирование создания объектов подклассам.

Примеры в Qt:

- `QStyle` — создаёт платформозависимые виджеты;
- `QIcon` — загружает иконки в разных форматах;
- фабрики для плагинов и расширений.

5. Singleton (одиночка)

Где используется: глобальные менеджеры и сервисы.

Примеры:

- `QApplication` — единственный экземпляр приложения;
- `QSettings` — доступ к настройкам системы;
- менеджеры шрифтов (`QFontDatabase`).

6. Adapter (адаптер)

Где используется: интеграция сторонних библиотек и данных.

Описание

Преобразование интерфейса одного класса к другому.

В Qt:

- адаптеры для работы с базами данных (SQL-модели);
- конвертация форматов данных (например, JSON/XML → модель).

7. Strategy (стратегия)

Где используется: динамическая смена поведения.

Примеры:

- QPainter с разными QPen/QBrush — выбор стратегии отрисовки;
- QSortFilterProxyModel — настраиваемая сортировка и фильтрация.

8. Composite (компоновщик)

Где используется: иерархия виджетов и графики.

Описание

Объединение объектов в древовидные структуры.

В Qt:

- иерархия виджетов (QWidget — родитель/потомок);
- графика в QGraphicsScene (элементы, группы, сцены).

9. Command (команда)

Где используется: отмена/повтор действий.

Примеры:

- QUndoStack — стек команд для отмены;
- редакторы с поддержкой Ctrl+Z/Ctrl+Y.

10. Active Object (активный объект)

Где используется: асинхронные операции и многопоточность.

Компоненты:

- прокси-объект (QObject);
- очередь сообщений;
- рабочий поток.

В Qt:

- QtConcurrent — высокоуровневое управление потоками;
- QFuture/QFutureWatcher — отслеживание асинхронных задач;

- цепочки продолжений через `QFuture::then()` (Qt6).

Специфические паттерны Qt6

В QML/Qt Quick

- **Declarative UI** — декларативное описание интерфейса (аналогично React/Vue).
- **State-Transitions** — управление состояниями и анимациями (паттерн State Machine).
- **Loader** — динамическая загрузка компонентов (Lazy Load).
- **StackView** — навигация между страницами (паттерн Stack).

В многопоточности (Qt Concurrent)

- **Future/Promise** — `QFuture` для асинхронных вычислений.
- **Thread Pool** — `QThreadPool` для управления потоками.
- **Map-Reduce** — параллельная обработка коллекций через `QtConcurrent::map()`.

В графике (Qt 3D, QGraphicsView)

- **Scene Graph** — оптимизация отрисовки сложных сцен.
- **Visitor** — обход иерархии элементов (например, для рендеринга).